A Heuristic Algorithm for the Inventory Routing Problem with Logistic Ratio

March 2024

1 Abstract

The paper presents a multi-start iterated local search for solving the inventory routing problem with logistic ratio. The proposed optimisation algorithm utilise serval components include an application of the Lagrangian relaxation where violation on the stockout constratins is permitted, a devliery quantity reoptimisation procedure, and serval local search operators integrates both routing decision and quantity decision. The computational experiements were conducted on 978 small instances with 5 to 50 customer and 150 large instances with 50 to 200 customers. The results shown the proposed algorithm outperformance all instances on previous available metaheuristics for large instances and produces competitive results on small instances. We also give our attempt on integrating the hybrid genetic search from capacited vehicle routing problem into the proposed multi-started iterated local search. With hybrid genetic search, the solution quality can still be improved by certain percentage.

2 Introduction

The inventory routing problem (IRP) arises in vendor-managed inventory systems, in which a supplier monitors the stock levels of its customers and takes responsibility for replenishing them. Over a multi-period planning horizon, the supplier jointly decides when each customer is visited, how much product is delivered at each visit, and how the vehicles are routed, subject to customer storage capacities, vehicle capacities, the supplier's available inventory, and the requirement that no customer runs out of stock. Integrating these decisions, rather than fixing replenishment plans first and routing them afterwards, can produce substantial savings (Archetti and Speranza, 2016). However, the integration couples a vehicle routing problem in every period with inventory dynamics that link the periods together. As a result, the IRP remains one of the most challenging problems in distribution logistics (Moin and Salhi, 2007; Bertazzi and Speranza, 2012; Coelho et al., 2014; Ekşi et al., 2026).

Most of the IRP literature minimises the total cost, defined as the sum of routing and inventory holding costs. In many applications, however, holding costs are difficult to estimate or are negligible compared with transport costs, and practitioners evaluate distribution plans by the transport cost per unit of product delivered. This key performance indicator, known as the *logistic ratio*, has long been used in the distribution of industrial gases (Bell et al., 1983; Benoist et al., 2011; Singh et al., 2015). It was also the objective of the ROADEF/EURO Challenge 2016, proposed by Air Liquide (André et al., 2020). Archetti et al. (2017b) formalised the IRP with logistic ratio (IRP-LR) and showed that solutions that minimise total cost can be poor in terms of the logistic ratio. One reason is the finite-horizon effect: under a cost objective there is little incentive to deliver product near the end of the horizon if it will only be consumed afterwards. The logistic ratio, in contrast, rewards every unit delivered and therefore produces replenishment plans that differ markedly from cost-minimising ones.

The IRP-LR is NP-hard in the strong sense, since it contains the travelling salesman problem as a special case (Section 3.1). Its objective is moreover the ratio of two decision-dependent quantities: the routing cost in the numerator and the total delivered quantity in the denominator. Exact approaches build on parametric fractional programming (Dinkelbach, 1967), which replaces the ratio with a sequence of linear objectives. Archetti et al. (2017b) and Archetti et al. (2019) developed exact algorithms for the IRP-LR along these lines, and Baldacci et al. (2020) proposed a general exact framework for vehicle routing problems with fractional objectives. These methods provide optimal solutions and bounds, but their computational requirements restrict them to instances with at most 50 customers. For larger instances, to the best of our knowledge, the only heuristics evaluated on the IRP-LR benchmark are the iterated local search (ILS) and simulated annealing (SA) algorithms of Alvarez et al. (2018). The algorithms developed for the ROADEF/EURO Challenge 2016 address a richer industrial variant with a ratio objective, using heuristic branch-cut-and-price (Absi et al., 2020), local search combined with mathematical programming (Su et al., 2020), fixed-sequence reoptimisation of quantities and timing (He et al., 2020), and hyper-heuristic sequence selection (Kheiri, 2020). More recently, ratio objectives have been studied in a multi-source setting solved by ILS (Alonso-Pecina et al., 2024) and in a multi-product setting with lateral transshipment and uncertain demand (Man et al., 2025). Compared with the classical IRP, for which a wide range of exact and heuristic methods exists, the heuristic literature on the IRP-LR remains sparse.

2.1 Related literature and research gaps

For the classical IRP, exact methods based on branch-and-cut (Archetti et al., 2007; Coelho and Laporte, 2013), branch-price-and-cut (Desaulniers et al., 2016), and their matheuristic extensions (Skålnes et al., 2023) solve instances of moderate size. Large instances are handled by heuristics, which are reviewed by Sofianopoulou and Mitsopoulos (2021) and Ekşi et al. (2026). We organise the

discussion around three design choices that are particularly important under a logistic-ratio objective. Table 2 summarises the studies most closely related to this work.

Where delivery quantities are decided. Efficient local search is available for routing decisions, whereas delivery quantities are naturally determined by mathematical programming. Most IRP heuristics therefore decompose the problem: a heuristic explores visit schedules and routes, and a mathematical program sets or repairs the quantities (Iswari et al., 2026). Examples include the tabu search combined with mixed-integer programming of Archetti et al. (2012), the matheuristic of Archetti et al. (2017a), the variable MIP neighbourhood descent of Coelho et al. (2020), the ILS matheuristic of Laganà et al. (2024), and route-improving matheuristics for related production routing problems (Vadseth et al., 2023). In the hybrid genetic search (HGS) of Castro et al. (2026), the routing component builds on the HGS for the capacitated vehicle routing problem (CVRP) (Vidal, 2022), while the quantities are obtained by solving a minimum-cost flow problem with the network simplex algorithm. Zhao et al. (2025) extend HGS with a large neighbourhood that simultaneously changes visit periods, insertion positions, and delivery quantities across the horizon. Decomposition works well when holding costs make quantity decisions secondary to routing. Under the logistic ratio, however, the change in delivered quantity caused by a routing move is part of the value of that move. A move that is evaluated with quantities held fixed can appear unattractive even though it becomes improving once the quantities are adjusted, and solving a mathematical program for every candidate move is too expensive within a local search. This suggests evaluating each move by its combined effect on routing cost and delivered quantity. It also suggests using an exact optimisation step for quantities only occasionally, as He et al. (2020) showed with fixed-sequence reoptimisation.

How infeasibility is handled. Allowing a search to visit infeasible solutions and penalising the violation in the objective is a well-established strategy in vehicle routing. It is a central component of HGS (Vidal, 2022), and in inventory routing it has been used to relax vehicle capacity (Archetti et al., 2012; Castro et al., 2026) and customer stockouts (Zhao et al., 2025). In the IRP, the stockout constraints make the feasible region particularly fragmented. Removing a single visit usually causes a stockout in some later period, so a search restricted to feasible solutions cannot easily move between replenishment plans with different visit patterns. Penalising stockouts addresses this, but under a ratio objective the penalty weight is difficult to set. The logistic ratio is measured in cost per unit and varies widely across instances, so a fixed penalty is either too weak, and the search remains infeasible, or too strong, and the search behaves as if infeasibility were forbidden. Lagrangian relaxation (Fisher, 1981) offers a principled alternative: the stockout constraint is dualised, and the multiplier is updated with a subgradient-type rule that scales the penalty to the current gap

from the incumbent. To the best of our knowledge, this has not previously been used to guide a local search for the IRP-LR.

Saturated vehicles. Because the logistic ratio rewards large deliveries, vehicles in good IRP-LR solutions are typically fully loaded. Any move that inserts a customer into such a vehicle is then infeasible if the existing quantities are kept, so large parts of the inter-route and inter-period neighbourhoods are effectively unavailable. In the CVRP, demands are fixed, so a move into a full vehicle can only be retained by violating its capacity at a penalty, as in HGS (Vidal, 2022). In the IRP-LR, delivery quantities are themselves decisions, which allows a different remedy that keeps every solution capacity-feasible. The deliveries already on a full vehicle often include quantities that are not needed to prevent future stockouts, and this slack can be reassigned to the inserted customer without violating any constraint. Exploiting this slack keeps the search within capacity while giving access to the moves that matter most for the logistic ratio.

Together, these observations identify the gap addressed in this paper: there is no scalable heuristic for the IRP-LR that (i) evaluates routing and quantity decisions jointly within each move, (ii) explores inventory-infeasible solutions in a controlled manner that adapts to the ratio objective, and (iii) does not depend on a mathematical programming solver.

2.2 Contributions

This paper proposes a multi-start iterated local search (MSILS) for the IRP-LR. The algorithm does not require a mathematical programming solver. Its main contributions are as follows.

1. **Joint evaluation of route and quantity decisions.** Four of the five local search operators update delivery quantities as part of each move, using closed-form rules that respect depot supply, vehicle capacity, and customer storage capacity. Every candidate move is therefore evaluated by its actual effect on the logistic ratio, that is, on routing cost and delivered quantity together.

2. **Vehicle slack.** We introduce a mechanism that reassigns quantity not needed to prevent future stockouts from the existing deliveries of a fully loaded vehicle to a newly inserted customer. Unlike the penalised capacity violations used in routing heuristics such as HGS (Vidal, 2022), which are needed when demands are fixed, the mechanism exploits the flexibility of IRP delivery quantities and never violates vehicle capacity. It makes available the inter-route and inter-period moves that would otherwise be rejected, and it creates no new stockouts.

3. **Lagrangian-guided exploration of infeasible solutions.** The zero-stockout constraint is dualised, and the local search solves a sequence of relaxed problems in which the Lagrange multiplier is updated by a subgradient-type rule based on the gap from the incumbent. The penalty

Table 1: Studies most relevant to the IRP-LR and to the proposed solution methodology.

Study	**Problem / objective**	**Solution approach**	**Key search characteristic**	**Relevance to this work**
Archetti et al. (2017b)	IRP; routing cost / quantity delivered	Exact fractional optimisation	Joint routing and replenishment	Introduces the IRP-LR
Alvarez et al. (2018)	IRP including logistic ratio	ILS and SA	Neighbourhood search over routes and replenishment	Benchmark heuristic for the IRP-LR
Archetti et al. (2019)	IRP-LR	Iterative exact method	Sequence of IRPs with linear objectives	Optimal solutions for small instances
Baldacci et al. (2020)	VRP with fractional objective	Exact method	Primal and dual bounding procedures	General fractional routing methodology
Absi et al. (2020)	Rich IRP; unit distribution cost	Heuristic branch-cut-and-price	Integrated routing, quantity, and scheduling	Large-scale industrial ratio IRP
Su et al. (2020)	ROADEF/EURO IRP	Local search with MIP/LP	Mathematical programming sets timing and quantities	Hybrid local-search design
He et al. (2020)	ROADEF/EURO IRP	Fixed-sequence matheuristic	Reoptimises quantities and timing for fixed visit sequences	Separation of route structure and quantity optimisation
Alonso-Pecina et al. (2024)	Operating cost / quantity delivered	ILS	Neighbourhood search for a multi-source IRP	Recent ILS with a ratio objective
Man et al. (2025)	Multi-product logistics-ratio IRP	Robust fractional optimisation	Uncertainty and lateral transshipment	Extension of logistics-ratio optimisation
Castro et al. (2026)	Classical IRP	HGS	Route search with min-cost-flow quantity optimisation	State-of-the-art decomposition-based search
Zhao et al. (2025)	Classical and stockout-penalised IRP	HGS with large neighbourhood	Cross-period route/quantity moves; penalised stockouts	Joint moves and controlled infeasibility
This paper	IRP-LR	Multi-start ILS (solver-free)	Joint route–quantity moves; vehicle slack; Lagrangian stockout penalty	Scalable heuristic for the IRP-LR

therefore scales automatically to the magnitude of the logistic ratio of each instance, allowing the search to cross infeasible regions and then return to feasibility.

4. **Problem-specific construction, rebalancing, and perturbation.** Initial solutions are built by a randomised constructive heuristic that prioritises customers by stockout urgency and distance from the depot. A rebalance procedure redistributes quantities over a fixed schedule whenever the incumbent improves, and an adaptive perturbation increases its strength as the number of non-improving iterations grows.

5. **Computational evidence.** Experiments on 1128 benchmark instances, comprising 978 instances with 5 to 50 customers and 150 instances with 50 to 200 customers, show that the MSILS improves on the ILS and SA of Alvarez et al. (2018) by on average 6.30% and 8.00%, respectively, on the large instances, with larger gains as the instance size grows. On small instances, it finds solutions within X% of the optimal values reported by Archetti et al. (2019). A component analysis quantifies the contribution of each mechanism, and we also examine whether embedding the HGS of Vidal (2022) as a routing post-optimiser brings further improvement. The instances are publicly available (and the code will be released upon acceptance).

Preliminary results of this work were presented at the 15th International Conference on Operations Research and Enterprise Systems (ICORES 2026) [add citation]. The present paper extends that work with [list extensions, e.g. the complete algorithm description, the component analysis, and the experiments on the large instances].

The remainder of the paper is organised as follows. Section 3 defines the IRP-LR and the Lagrangian-relaxed problem used by the algorithm. Section 4 describes the MSILS. Section 5 reports the computational experiments, and Section **??** concludes and outlines directions for future research.

3 Problem Background and Related Literature

The Inventory Routing Problem (IRP) integrates inventory replenishment and vehicle-routing decisions over a multi-period planning horizon. Rather than taking delivery quantities as predetermined inputs, the supplier jointly determines when customers are visited, how much product is delivered, and how the corresponding vehicle routes are constructed (Bertazzi and Speranza, 2012; Coelho et al., 2014). This integration can reduce system-wide logistics costs, but it also creates strong dependencies between routing decisions, delivery quantities, and customer inventory levels across periods.

Most classical IRP formulations minimise an absolute measure of routing, inventory, or total logistics cost. Archetti et al. (2017b) introduced an alternative performance measure, referred to as the *logistic ratio*, defined as the

routing cost per unit of product delivered. Their motivation arises partly from the finite-horizon effect of conventional IRP objectives. Under total-cost minimisation, there is generally little incentive to deliver additional inventory near the end of the planning horizon if it is consumed only after the horizon. A cost-per-unit-delivered objective explicitly accounts for the amount transported and can therefore mitigate this effect.

For a solution s, let $C(s)$ denote its total routing cost and $Q(s)$ the total quantity delivered. The logistic ratio is

$$LR(s) = \frac{C(s)}{Q(s)}. \tag{1}$$

If $Q(s)$ were constant across all feasible solutions, minimising (1) would be equivalent to minimising routing cost. In an IRP, however, delivery quantities are endogenous. Consequently, the objective simultaneously favours

$$C(s) \downarrow \qquad \text{and} \qquad Q(s) \uparrow,$$

and may therefore generate substantially different replenishment decisions from a conventional cost-minimising IRP.

The problem studied in this paper follows the single-product IRP with logistic ratio (IRP-LR) introduced in Archetti et al. (2017b). The formulation below additionally introduces auxiliary stockout variables, denoted by μ_{it}, which are forced to zero in the original problem and therefore do not modify its feasible solution set. Their purpose is to enable the Lagrangian-relaxation mechanism used later within the proposed solution algorithm.

Two properties of the problem shape the design of the algorithm. First, a feasible solution is not difficult to obtain: visiting a customer in every period prevents it from running out of stock. The difficulty therefore lies not in reaching feasibility but in finding a plan that keeps the routing cost low while delivering as much product as possible. Second, the production at the depot is sufficient to meet the demand of all customers over the planning horizon.

3.1 Problem Formulation

Let $G = (N, E)$ be a complete undirected graph, where $N = \{0\} \cup N'$ is the set of locations and E is the set of edges. Node 0 represents the depot and $N' = \{c_1, c_2, \ldots, c_{|N'|}\}$ denotes the set of customers. A non-negative routing cost c_{ij} is associated with each edge $\langle i, j\rangle \in E$, and the costs are assumed to satisfy the triangle inequality. The planning horizon $T = \{1, \ldots, |T|\}$ consists of discrete periods. Each customer $i \in N'$ has an inventory capacity U_i and consumes r_{it} units in period $t \in T$. The depot produces r_{0t} units in period t, and I_{i0} denotes the initial inventory at location $i \in N$. A fleet $K = \{k_1, \ldots, k_{|K|}\}$ of homogeneous vehicles is available, each with capacity Q.

The objective is to construct a distribution plan that minimises the logistic ratio while ensuring that vehicle capacities and customer inventory capacities are respected and that no stockout occurs in the original problem. Split deliveries

are not permitted: a customer can be served by at most one vehicle in each period. The following decision variables are used:

- $I_{it} \geq 0$: inventory level at location i at the end of period t;
- y_{ij}^{kt}: number of times edge $\langle i,j \rangle$ is traversed by vehicle k in period t;
- $q_{it}^{k} \geq 0$: quantity delivered to customer i by vehicle k in period t;
- $z_{it}^{k} \in \{0,1\}$: equal to one if location i is visited by vehicle k in period t;
- $\mu_{it} \geq 0$: auxiliary stockout quantity associated with customer i in period t.

For a subset $S \subseteq N'$, define $E(S) = \{\langle i,j \rangle \in E : i,j \in S\}$. Assuming that the total delivered quantity is positive, the IRP-LR is formulated as follows:

$$\min \quad \frac{\sum_{k\in K}\sum_{\langle i,j\rangle\in E}\sum_{t\in T} c_{ij} y_{ij}^{kt}}{\sum_{k\in K}\sum_{i\in N'}\sum_{t\in T} q_{it}^{k}} \tag{2}$$

$$\text{s.t.} \quad I_{0t} = I_{0,t-1} + r_{0t} - \sum_{k\in K}\sum_{i\in N'} q_{it}^{k}, \qquad t \in T, \tag{3}$$

$$I_{it} - \mu_{it} = I_{i,t-1} - \mu_{i,t-1} - r_{it} + \sum_{k\in K} q_{it}^{k}, \quad i \in N',\ t \in T, \tag{4}$$

$$I_{it} \geq 0, \qquad i \in N,\ t \in T, \tag{5}$$

$$\mu_{it} \geq 0, \qquad i \in N',\ t \in T, \tag{6}$$

$$\sum_{i\in N'}\sum_{t\in T} \mu_{it} \leq 0, \tag{7}$$

$$\sum_{k\in K} q_{it}^{k} \leq U_i - I_{i,t-1}, \qquad i \in N',\ t \in T, \tag{8}$$

$$q_{it}^{k} \leq U_i z_{it}^{k}, \qquad i \in N',\ k \in K,\ t \in T, \tag{9}$$

$$\sum_{i\in N'} q_{it}^{k} \leq Q z_{0t}^{k}, \qquad k \in K,\ t \in T, \tag{10}$$

$$\sum_{k\in K} z_{it}^{k} \leq 1, \qquad i \in N',\ t \in T, \tag{11}$$

$$\sum_{j:\langle i,j\rangle\in E} y_{ij}^{kt} = 2 z_{it}^{k}, \qquad i \in N,\ k \in K,\ t \in T, \tag{12}$$

$$\sum_{\langle i,j\rangle\in E(S)} y_{ij}^{kt} \leq |S| - 1, \qquad S \subseteq N',\ |S| \geq 2,\ k \in K,\ t \in T, \tag{13}$$

$$z_{it}^{k} \in \{0,1\}, \qquad i \in N,\ k \in K,\ t \in T, \tag{14}$$

$$q_{it}^{k} \geq 0, \qquad i \in N',\ k \in K,\ t \in T, \tag{15}$$

$$y_{ij}^{kt} \in \{0,1,2\}, \qquad \langle i,j\rangle \in E,\ k \in K,\ t \in T. \tag{16}$$

The objective function (2) minimises total routing cost per unit of product delivered. Constraints (3) and (4) impose the inventory balance equations at the depot and customers, respectively. For a customer, $I_{it} - \mu_{it}$ represents its net inventory position. Since both I_{it} and μ_{it} are non-negative, Constraint (7) forces $\mu_{it} = 0$ for every customer and period in the original problem and hence eliminates stockouts.

Constraint (8) prevents deliveries from exceeding the available customer storage capacity, while Constraint (9) allows a positive delivery only if the corresponding customer is visited. Constraint (10) imposes vehicle capacity, and Constraint (11) prevents split deliveries within a period. Constraints (12) and (13) ensure valid vehicle routes. Because the number of subtour-elimination constraints is exponential, Constraint (13) can be generated dynamically when the formulation is solved exactly. Constraints (14)–(16) specify the remaining variable domains.

The IRP-LR is NP-hard in the strong sense, even with a single period and a single vehicle. Consider an instance with $|T| = |K| = 1$ in which every customer $i \in N'$ has zero initial inventory, $I_{i0} = 0$, a positive demand $r_{i1} > 0$ and capacity $U_i = r_{i1}$, and in which $Q \geq \sum_{i \in N'} r_{i1}$ and $I_{00} + r_{01} \geq \sum_{i \in N'} r_{i1}$. By Constraints (4)–(8), every customer must be visited and receives exactly r_{i1}, so the total quantity delivered equals $\sum_{i \in N'} r_{i1}$ in every feasible solution. Minimising the logistic ratio is then equivalent to minimising the routing cost of a single route through all customers, that is, to solving a travelling salesman problem, which is NP-hard in the strong sense. Consequently, exact methods can be expected to be limited to small instances, and heuristics are required for instances of practical size.

3.2 Solution Approaches in the Literature

The fractional objective adds an additional layer of difficulty to the already challenging interaction between routing, inventory, and replenishment quantities. The IRP-LR belongs to the broader class of fractional optimisation problems. The classical parametric approach of Dinkelbach (1967) transforms a fractional problem into a sequence of optimisation problems with an objective of the form

$$N(s) - \lambda D(s).$$

This connection has motivated exact approaches for routing problems with decision-dependent ratios.

Following the introduction of the IRP-LR, Archetti et al. (2019) developed an improved exact algorithm that solves the fractional problem through a sequence of IRPs with linear objective functions. Baldacci et al. (2020) subsequently investigated fractional objectives in vehicle routing more generally and developed an exact framework based on specialised primal and dual bounds.

Metaheuristic approaches are particularly relevant for larger instances. Alvarez et al. (2018) proposed Iterated Local Search (ILS) and Simulated Annealing algorithms for the IRP and considered the logistic-ratio objective, providing

an important heuristic precedent for the present work. The ROADEF/EURO Challenge 2016 further stimulated several algorithms for a rich industrial inventory routing problem with a unit-distribution-cost objective. These include heuristic branch-cut-and-price (Absi et al., 2020), local search combined with mathematical programming (Su et al., 2020), fixed-sequence reoptimisation (He et al., 2020), and hyper-heuristic search (Kheiri, 2020).

Of particular relevance, He et al. (2020) show that substantial improvements can be obtained by reoptimising delivery quantities and timing while retaining a fixed customer sequence. This distinction between route structure and quantity optimisation is especially important under a logistic-ratio objective because quantities directly affect the denominator of the objective.

More recently, Alonso-Pecina et al. (2024) proposed an ILS for a multiple-source inventory-routing application in which operating cost is divided by total delivered quantity. Man et al. (2025) extended logistics-ratio optimisation to a multi-product setting with lateral transshipment and uncertain demand.

Recent developments for conventional cost-based IRPs are also methodologically relevant. Castro et al. (2026) developed a Hybrid Genetic Search (HGS) that combines population-based search, specialised neighbourhoods, and an inventory-flow optimisation procedure. Zhao et al. (2025) proposed a large neighbourhood within HGS that jointly changes visit periods, routing insertions, and delivery quantities across the planning horizon. Their work also considers controlled inventory infeasibility through penalised stockouts. Although these recent HGS approaches do not optimise the same logistic-ratio formulation considered here, they highlight the potential benefits of coupling powerful route exploration with specialised quantity optimisation and of allowing controlled infeasibility during the search.

3.3 Lagrangian-Relaxed Problem

The literature suggests two particularly useful principles for difficult IRPs: delivery quantities should be reconsidered jointly with the route structure, and restricting the search to feasible solutions can limit the ability to move between substantially different replenishment plans. The proposed algorithm exploits both ideas.

In the original formulation, Constraint (7), together with $\mu_{it} \geq 0$, forces all stockout variables to zero. To allow the search to temporarily explore inventory-infeasible solutions, we dualise Constraint (7) using a Lagrange multiplier $\alpha \geq 0$. Following the general principle of Lagrangian relaxation (Fisher, 1981), the resulting relaxed objective is

$$\min \quad \frac{\sum_{k \in K} \sum_{\langle i,j \rangle \in E} \sum_{t \in T} c_{ij} y_{ij}^{kt}}{\sum_{k \in K} \sum_{i \in N'} \sum_{t \in T} q_{it}^{k}} + \alpha \sum_{i \in N'} \sum_{t \in T} \mu_{it}. \tag{17}$$

Table 2: Selected literature relevant to the IRP-LR and the proposed solution methodology.

Study	**Problem / objective**	**Solution approach**	**Key search characteristic**	**Relevance**
Archetti et al. (2017b)	IRP; routing cost / quantity delivered	Exact fractional optimisation	Joint routing and replenishment optimisation	Introduces and analyses the IRP-LR
Alvarez et al. (2018)	IRP including logistic-ratio objective	ILS and Simulated Annealing	Neighbourhood-based routing and replenishment search	Direct metaheuristic precedent for IRP-LR
Archetti et al. (2019)	IRP-LR	Iterative exact method	Sequence of IRPs with modified linear objectives	Main exact IRP-LR methodology
Absi et al. (2020)	Rich IRP; unit distribution cost	Heuristic branch-cut-and-price	Integrated routing, quantity and scheduling optimisation	Large-scale fractional industrial IRP
Su et al. (2020)	ROADEF/EURO IRP	Local search with MIP/LP	Mathematical programming improves timing and quantities	Relevant hybrid local-search approach
He et al. (2020)	ROADEF/EURO IRP	Fixed-sequence matheuristic	Reoptimises quantities and timing for fixed visit sequences	Closest precedent for route-fixed quantity optimisation
Baldacci et al. (2020)	Fractional VRP	Exact fractional-routing method	Primal and dual bounding procedures	General fractional-routing methodology
Alonso-Pecina et al. (2024)	Operating cost / quantity delivered	Iterated Local Search	Realistic inventory-routing neighbourhood search	Recent ILS with a ratio objective
Man et al. (2025)	Multi-product logistics-ratio IRP	Robust fractional optimisation	Uncertainty and lateral transshipment	Recent extension of logistics-ratio optimisation
Castro et al. (2026)	Conventional IRP	Hybrid Genetic Search	Route search combined with inventory-flow optimisation	Recent high-performance IRP search framework
Zhao et al. (2025)	Conventional and stockout-penalised IRP	HGS and large-neighbourhood search	Cross-period route/quantity optimisation and controlled infeasibility	Highly relevant search-design principles

The relaxed problem consists of objective (17) subject to

$$(3) - -(6), \qquad (8) - -(16),$$

that is, Constraint (7) is removed. The variables μ_{it} can therefore take positive values and quantify inventory shortages, which are penalised through the multiplier α.

A small value of α allows the search to explore solutions with larger stock-out violations when this facilitates improvements in routing and replenishment structure, whereas a larger value increasingly discourages such violations and directs the search towards the feasible region. Thus, the relaxed formulation provides a mechanism for controlled exploration of both feasible and infeasible regions of the solution space.

In what follows, formulation (2)–(16) is referred to as the *original problem*, while (17) together with the remaining constraints is referred to as the *relaxed problem*. The proposed MSILS algorithm uses the relaxed problem during local search and dynamically adjusts the multiplier α, as described in Section 4.

4 Description of the Algorithm

This section describes the proposed Multi-Start Iterated Local Search (MSILS) algorithm for solving the Inventory Routing Problem with Logistic Ratio (IRP-LR). The algorithm uses features that include a mechanism for delivery quantity adjustment, delivery quantity-focused local search operators, and exploring infeasible regions through the amalgamation of iterated local search and Lagrangian relaxation. The remainder of this section is organised as follows. Section 4.1 presents the overall MSILS framework. Section 4.2 describes the constructive heuristic used to generate the initial solutions. Section 4.3 provides the details on the local search procedure for solving the relaxed problem. Section 4.4 introduces a quantity-focused rebalancing procedure that attempts to improve total delivery quantity while preserving route schedules. Finally, Section 4.5 describes the perturbation mechanism used to diversify the search and escape local optima.

4.1 Main algorithm

Let s denote a solution, which may be infeasible for the original problem (2)-(16). For each customer $i \in N'$, each time period $t \in T$, and each vehicle $k \in K$, part of the solution s is characterised by $q_{it}^k(s)$, $I_{it}(s)$, and $\mu_{it}(s)$, where

- $q_{it}^k(s) \geq 0$ is the delivery quantity by vehicle k, to customer i in time period t;
- $I_{0t}(s) \geq 0$ is the inventory level at the depot in time period t;
- $I_{it}(s) \geq 0$ is the inventory level at customer i in time period t; and

- $\mu_{it}(s) \geq 0$ is the stockout amount at customer i in time period t.

If customer i is visited by vehicle k in time t, $q_{it}^k(s)$ is positive; zero otherwise. The net inventory level is defined as $I_{it}(s) - \mu_{it}(s)$ and is computed recursively according to

$$I_{i,1}(s) - \mu_{i,1}(s) = I_{i,0} - r_{i,1} + \sum_{k \in K} q_{i1}^k(s), \tag{18}$$

$$I_{i,t}(s) - \mu_{i,t}(s) = I_{i,t-1}(s) - \mu_{i,t-1}(s) - r_{i,t} + \sum_{k \in K} q_{it}^k(s),\ 2 \leq t \leq |T|. \tag{19}$$

The inventory level of the depot is computed by

$$I_{0,1}(s) = I_{0,0} + r_{0,1} - \sum_{i \in N'} \sum_{k \in K} q_{i1}^k(s), \tag{20}$$

$$I_{0,t}(s) = I_{0,t-1}(s) + r_{0,t} - \sum_{i \in N'} \sum_{k \in K} q_{it}^k(s),\ 2 \leq t \leq |T|. \tag{21}$$

If s is feasible to the relaxed problem and has $\sum_{i \in N'} \sum_{i \in T} \mu_{it}(s) = 0$, then s is also feasible for the original problem. The value of s on the objective function (2) is denoted by $f(s)$, and the value on the objective function (17) with Lagrange multiplier α is denoted by $f_\mu(s, \alpha)$. The following pseudocode outlines the proposed MSILS.

Algorithm 1 MSILS

1: $s^* \leftarrow \emptyset$, $f(s^*) \leftarrow +\infty$
2: **for** $i \leftarrow 1$ **to** $Max_{restart}$ **and** elapsed time $< Max_{time}$ **do**
3: $s \leftarrow$Initial_Solution_Procedure, $\hat{s} \leftarrow s$, $it \leftarrow 0$
4: **if** $s = \emptyset$ **then**
5: **return** s^*
6: **end if**
7: **if** $f(s) < f(s^*)$ **then**
8: $s^* \leftarrow s$
9: **end if**
10: **while** $it < \lceil \zeta(|N'| + \sigma|T| + \phi|K|) \rceil$ **do**
11: $\alpha \leftarrow \epsilon(s^*, s, 0, \eta)$, $\bar{s} \leftarrow s$, $s \leftarrow$Local_Search(s, α)
12: **while** $f_\mu(s, \alpha) \neq f_\mu(\bar{s}, \alpha)$ **or** s has stockout **do**
13: $\alpha \leftarrow \epsilon(s^*, s, \alpha, \eta)$, $\bar{s} \leftarrow s$, $s \leftarrow$Local_Search(s, α)
14: **end while**
15: **if** s is feasible **and** $f(s) < f(\hat{s})$ **then**
16: $\hat{s} \leftarrow s$, $it \leftarrow 0$, $\hat{s} \leftarrow$Rebalance$(\hat{s})$
17: **end if**
18: **if** $\hat{s}$ is feasible **and** $f(\hat{s}) < f(s^*)$ **then**
19: $s^* \leftarrow \hat{s}$
20: **end if**
21: $s \leftarrow$ Disturbance$(\hat{s}, it)$, $it \leftarrow it + 1$
22: **end while**
23: **end for**
24: **return** s^*

Initially, the global best-known solution s^* is empty, and $f(s^*)$ is set to $+\infty$ (line 1). s^* is updated throughout the algorithm. The FOR loop (lines 2-23) implements the multi-start framework. At each restart, a feasible solution to the original problem is generated using the constructive heuristic described in 4.2. This solution is used as the starting solution for the upcoming application of the iterated local search (WHILE loop, lines 10-22). By repeatedly restarting the iterated local search from different initial solutions, the algorithm can explore diversified regions of the solution space. The number of starting solutions is controlled by the parameter $Max_{restart}$. The algorithm also terminates once the elapsed computational time reaches the time limit Max_{time}; this limit is checked throughout the search, so an ongoing iterated local search is interrupted when it is reached, and the best solution s^* found so far is returned.

Each application of the iterated local search uses a local search procedure to solve a sequence of relaxed problems, each with a different value of α. If an optimal solution $\tilde{s}$ for the relaxed problem can be obtained, then following a commonly used version of the Lagrangian relaxation method Fisher (1981), α is computed by

$$\alpha + \frac{\eta \times (f(s^*) - f_\mu(\tilde{s}, \alpha)) \times (\sum_{i \in N'} \sum_{t \in T} \mu_{it}(\tilde{s}))}{\left(\sum_{i \in N'} \sum_{t \in T} \mu_{it}(\tilde{s})\right)^2} \tag{22}$$

where η is a scalar. Since the local search cannot guarantee the optimal solution $\tilde{s}$, instead of (22), α is computed by

$$\epsilon(s^*, s, \alpha, \eta) = \alpha + \frac{\eta \times \delta \times (\sum_{i \in N'} \sum_{t \in T} \mu_{it}(s) + 0.001)}{\left(\sum_{i \in N'} \sum_{t \in T} \mu_{it}(s)\right)^2 + 0.00001} \tag{23}$$

where

$$\delta = \begin{cases} f(s^*) - f_\mu(s, \alpha) & \text{if } f(s^*) - f_\mu(s, \alpha) > 0; \\ 0.1 \times f(s^*) & \text{Otherwise.} \end{cases}$$

Once local search obtains a feasible solution for the original problem, $\epsilon(\cdot)$ becomes just

$$\alpha + \frac{\eta \times \delta \times (0.001)}{0.00001}$$

which greedily increases α until the stopping criterion is satisfied for the WHILE loop (lines 12-14). The initial value of α is computed by $\epsilon(s^*, s, 0, \eta)$ in line 11 and the subsequent values is computed by $\epsilon(s^*, s, \alpha, \eta)$ in line 13 using α from the previous call of the local search procedure.

To escape from the local optimum, the MSILS begins with a quantity-focused disturbance procedure, referred to as the rebalance procedure. This procedure attempts to further improve the locally best-known solution $\hat{s}$ by adjusting the delivery quantities, keeping the schedule fixed (see Section 4.4). The output of the procedure is either the input, meaning that the procedure failed to make any improvement or a feasible solution of the original problem with a lower logistic ratio. To limit computational overhead, the rebalance procedure is invoked only when $\hat{s}$ has been improved. To further diversify the search, the MSILS applies a route-focused disturbance procedure on $\hat{s}$, generating the starting solution for the subsequent call of local search. The iterated local search (WHILE loop lines 10-22) terminates after the counter it reaches $\lceil \zeta(|N'| + \sigma|T| + \phi|K|) \rceil$, the maximum permissible number of consecutive iterations without improvement, where σ and ϕ are the weight parameters on the number of time periods and the number of vehicles, respectively.

4.2 Initial feasible solution

This section describes the heuristic to construct the initial solution for the MSILS. It also aims to produce a good feasible solution to the original problem (2)-(16), thereby establishing the initial upper bound.

The solution s is initialised with empty routes and zero delivery quantities, $q_{it}^k(s) = 0$ for $i \in N'$, $k \in K$, and $t \in T$. The net inventory levels and the inventory levels at the depot are computed by (18)−(21). The procedure starts by grouping the customers according to the earliest time period in which their net inventory levels become negative. Those customers with earlier stockouts

are given higher priority. Within each stockout group, customers are further partitioned spatially. Let (x_i, y_i) denote the coordinates of location $i \in N$. Using the coordinate minima and maxima

$$x_{\min} = \min_{i \in N} x_i, \quad x_{\max} = \max_{i \in N} x_i,$$

$$y_{\min} = \min_{i \in N} y_i, \quad y_{\max} = \max_{i \in N} y_i,$$

an axis-aligned bounding rectangle is defined with corners (x_{min}, y_{min}), (x_{max}, y_{min}), (x_{max}, y_{max}), and (x_{min}, y_{max}). The resulting axis-aligned rectangle is divided into a $d \times d$ grid of subregions $R = \{r_1, r_2, \ldots, r_{d \times d}\}$ where the grid resolution d is selected uniformly at random from the Δ_d consecutive integers $\{d_{\min}, d_{\min} + 1, \ldots, d_{\min} + \Delta_d - 1\}$, with $d_{\min}$ the minimum grid resolution and Δ_d the grid resolution increment.

Observe that, during the early stages of the solution construction, vehicles typically have sufficient capacity to deliver the largest possible quantities respecting both vehicle and inventory capacity constraints. Such flexibility reduces as the construction goes on, and customers who will be considered at a later stage often require more visits. Thus, the subregions are ordered by decreasing distance from their centres to the depot, so customers in more distant subregions are processed earlier.

The priority grouping and the spatial ordering determine which customers are considered first, but the timing and size of each delivery must also respect the available supply. A delivery in period t reduces depot inventory in that period and in every subsequent period, according to (20) and (21). Therefore, the largest additional quantity that can be delivered in period t while keeping depot inventory non-negative is

$$S_t(s) = \max \left\{ 0, \min_{t' \in T:\, t' \geq t} I_{0,t'}(s) \right\}. \tag{24}$$

Thus, $S_t(s)$ is the supplier's residual capacity for an additional delivery in period t, accounting for its effect on depot inventory over the rest of the horizon.

For each stockout group and subregion, the heuristic randomly selects a customer and repeatedly schedules a visit to address that customer's earliest remaining stockout. It first searches for a period between the current search period and the stockout period in which the customer is not yet visited, the fleet has remaining capacity and $S_t(s) > 0$, choosing randomly among such periods. If none is available, it searches backwards for an earlier period satisfying the same conditions. In the selected period, it assigns the customer to the vehicle that can accommodate the largest delivery. The delivery quantity is bounded by $S_t(s)$, that vehicle's residual capacity, and the customer's remaining inventory capacity.

A new visit raises the customer's inventory in later periods, so the quantities of that customer's other visits may need adjustment. After each assignment, the procedure Adjust Delivery Quantities updates depot inventory and recomputes the customer's inventory trajectory in chronological order. Since the new visit

does not affect the periods before it, only the deliveries in the periods after the selected period are adjusted: it increases each existing delivery only up to the available supplier and vehicle capacity and the customer's remaining inventory capacity, and reduces it if necessary so that it does not exceed the customer's remaining inventory capacity. The heuristic continues until no customer stockout remains.

If no feasible visit period can be found while the customer still has a stockout, the construction stops and returns an empty solution, and the MSILS is terminated. In the computational experiments on the test bed described in Section 5.1, this case has never been observed.

Let $P_{t,r} \subseteq N'$ denote the subset of customers in priority group $t \in T$ and subregion $r \in R$. The pseudocode, Construct Initial Solution, summarises the procedure.

Construct Initial Solution

initialise solution s
partition customers by priority and distance to depot
for $t \in T$ **and** $r \in R$ **do**
  while $P_{t,r} \neq [\,]$ **do**
    randomly choose a customer i from $P_{t,r}$
    $t_{current} \leftarrow 1$
    while $\{\hat{t} \in T : I_{i,\hat{t}}(s) - \mu_{i,\hat{t}}(s) < 0\} \neq [\,]$ **do**
      $t_{stockout} \leftarrow \min\{t \in T : I_{i,t}(s) - \mu_{i,t}(s) < 0\}$
      $t_{assign} \leftarrow$ Find Feasible Visit Time $(t_{current}, t_{stockout}, i)$
      if $t_{assign} = \emptyset$ **then**
        return $\emptyset$
      end if
      $(k, q) \leftarrow$ Find Vehicle To Deliver Max(t_{assign}, i, s)
      $q^k_{i,t_{assign}} \leftarrow q$
      add customer i to vehicle k's route on time period t_{assign}
      Adjust Delivery Quantities(i, t_{assign}, s)
      $t_{current} \leftarrow t_{assign} + 1$
    end while
    $P_{t,r} \leftarrow P_{t,r} \setminus i$
  end while
end for
$s \leftarrow$nearest_neighbour_insertion(s)
return s

After the schedule has been constructed, a nearest-neighbour insertion Bräysy and Gendreau (2005) is applied independently to each vehicle's route in each period. It starts with the customer nearest to the depot. At each subsequent step, it selects the unvisited customer closest to the current tour and inserts that customer in the position that yields the smallest increase in the length of the closed route. The resulting solution is then returned to MSILS.

4.3 Local search

The local search is a sequential variable neighbourhood descent over five operators $O_1, \ldots, O_5$, applied to the relaxed problem with a given Lagrange multiplier α. The routing moves themselves are standard in vehicle routing (see, for example, Kindervater and Savelsbergh (1997), Laporte et al. (2000), and Elshaer and Awad (2020)) and inventory routing (see, for example, Alvarez et al. (2018), Laganà et al. (2024), and Castro et al. (2026)):

- O_1 moves up to 3 adjacent customers from one route to another route in the same period;
- O_2 exchanges up to 2 adjacent customers of one route with up to 2 adjacent customers of another route in the same period;
- O_3 transfers a customer visit to another period in which the customer is not visited, using the vehicle that can take the largest delivery and the insertion position of least cost;
- O_4 exchanges a visited customer with an unvisited one in the same route, including the cases where a visit is only removed or only inserted;
- O_5 moves up to 3 adjacent customers to another position in the same route.

The novelty of the local search lies in how operators O_1 to O_4 treat quantities. Heuristics for the IRP commonly evaluate routing moves with delivery quantities fixed and revise the quantities in a separate step, often by mathematical programming (Iswari et al., 2026). Here, each alteration determines both the routes and the resulting delivery quantities, so every move is evaluated by its actual effect on the logistic ratio, that is, on the transportation cost and the total quantity delivered at the same time. Operator O_5 is a pure routing operator and leaves quantities unchanged.

When an alteration removes the visit to customer $i \in N'$ in period $t \in T$, $q^k_{it}(s)$ becomes 0. When it creates a new visit by vehicle $k \in K$, the customer receives the largest quantity allowed by the depot supply, the residual capacity of the vehicle, and the customer's remaining inventory capacity,

$$q^k_{it}(s) = \min\Big\{ S_t(s),\ Q - \sum_{j \in N'} q^k_{jt}(s),\ U_i - \big(I_{i,t-1}(s) - \mu_{i,t-1}(s)\big) \Big\}, \tag{25}$$

where $I_{i,0}(s) - \mu_{i,0}(s) = I_{i,0}$. A change in period t alters the customer's net inventory in all later periods, so the customer's subsequent deliveries are revised in chronological order: each is increased as far as the residual capacity of its vehicle allows and reduced where necessary to respect U_i. Moves that would make the depot inventory negative in any period are discarded.

Vehicle slack. Because the logistic ratio rewards large deliveries, vehicles in good solutions are typically fully loaded. For a fully loaded vehicle, (25)

gives a zero quantity, so a large share of the moves that insert a customer into another route or period would be rejected. In the CVRP, where demands are fixed, such moves can only be retained by permitting capacity violations and penalising them in the objective, as in the hybrid genetic search of Vidal (2022). In the IRP-LR, delivery quantities are decisions, so the operators here instead borrow quantity that existing visits do not need, and vehicle capacity is never violated. The slack of the delivery to customer j in period t,

$$\omega_{jt}(s) = \max\Big\{0,\ \min\big\{\textstyle\sum_{k\in K} q^k_{jt}(s),\ \min_{t'\in T:\, t'\geq t}\big(I_{jt'}(s) - \mu_{jt'}(s)\big)\big\}\Big\}, \quad (26)$$

is the largest quantity that can be removed from this delivery without creating a stockout in period t or later. The slack of vehicle k in period t, $\Omega^k_t(s)$, is the sum of $\omega_{jt}(s)$ over the customers it serves in that period. If vehicle k has no residual capacity, the new visit to customer i receives

$$q^k_{it}(s) = \min\Big\{S_t(s),\ \Omega^k_t(s),\ U_i - \big(I_{i,t-1}(s) - \mu_{i,t-1}(s)\big)\Big\},$$

where the slack of any customers leaving vehicle k in the same move is excluded from $\Omega^k_t(s)$. The same amount is then removed from the deliveries of the other customers on vehicle k, each reduced by at most its slack. In O_3, if no vehicle has residual capacity in the target period, the vehicle with the largest slack is used. The move therefore keeps the vehicle within its capacity and creates no new stockout, while the load of the vehicle is redistributed among its customers. This enlarges the neighbourhoods of O_1 to O_4 to moves involving fully loaded vehicles, which would otherwise be excluded.

Local Search(s, α)

for $i \leftarrow 1$ **to** 5 **do**
  repeat
    $\hat{s} \leftarrow s$
    $s' \leftarrow \arg\min\{f_\mu(s'', \alpha) : s'' \in O_i(s)\}$
    if $f_\mu(s', \alpha) < f_\mu(s, \alpha)$ **then**
      $s \leftarrow s'$
    end if
  until $f_\mu(s, \alpha) = f_\mu(\hat{s}, \alpha)$
end for
$s \leftarrow$ remove any visit with zero delivery quantity
return s

The neighbourhoods are explored in the order O_1 to O_5. Within the neighbourhood $O_i(s)$, all neighbouring solutions are evaluated with $f_\mu(\cdot, \alpha)$, the best improving one is applied, and the neighbourhood is re-explored until no improving neighbour remains; the search then proceeds to the next operator. To save computation, pairs of routes for which no improving move was found are not re-evaluated until one of them changes, and O_5 is applied only to routes modified by O_1 to O_4. Finally, visits with zero delivery quantity are removed, and the resulting solution is returned.

4.4 Rebalance Procedure

The local search procedure simultaneously explores routing decisions and delivery quantities. However, the improved solution may still contain delivery quantities that are not fully consistent with the fixed schedule. In particular, some customer visits may carry quantities that are unnecessarily large, whereas other visits may have unused inventory or vehicle capacity that could be exploited to increase the total delivery quantity. Since the objective of the IRP-LR is expressed as the ratio between routing cost and delivery quantity, improving the quantity allocation alone can further improve the objective value even when the routing schedule remains unchanged. This section describes the rebalance procedure, which exploits such an opportunity whenever a new incumbent solution s^* is obtained.

The procedure consists of two phases. First, delivery quantities are partially removed from existing visits whenever such reductions do not create stockouts. This releases inventory and vehicle capacity that may have been inefficiently allocated. Second, the released quantities are reallocated among the existing visits. Customers are selected randomly, favouring customers with low inventory levels, high demand rates, long intervals until their next visits, and infrequent replenishment patterns. For each selected customer, the maximum feasible quantity is assigned while respecting inventory capacity and vehicle capacity. The inventory level is then updated, and visits associated with zero delivery quantity are subsequently removed.

By combining quantity reallocation with randomised customer selection, the rebalance procedure not only intensifies the search for better delivery quantities around a promising routing schedule but also introduces diversification, enabling the algorithm to escape local optima in the quantity domain. The pseudocode below summarises the rebalance procedure.

Algorithm 4 Rebalance(s^*)

```
1:  s ← s*
2:  for i ← 1 to |N'| do
3:     for t ← 1 to |T| do
4:        if customer i is visited on day t then
5:           Quantity ← Quantity - Feasible_Quantity_To_Removal(i, t)
6:           Update inventory levels for all subsequent days
7:        end if
8:     end for
9:  end for
10: s ←Assign_max_possible_quantity_after_feasible(s)
11: s ←Remove visits with zero delivery quantities
12: return s
```

In addition to the rebalance procedure described above, an alternative Destory_And_Rebalance procedure was developed. This variant first removes all delivery quantities and

then reconstructs them while keeping the routing schedule fixed. Because the reconstruction does not guarantee feasibility, a repair mechanism is subsequently applied to eliminate stockouts whenever possible. The larger perturbation induced by this procedure promotes diversification and allows the search to explore quantity allocations that are less accessible through the former rebalance procedure. The pseudocode for this variant is presented in Appendix D. Both rebalance procedures are examined in the computation experiments.

4.5 Perturbation

The perturbation procedure is a diversification mechanism designed to help the MSILS escape local optima. It randomly removes two or more adjacent customers from a route together with their associated delivery quantities, after which inventory levels and subsequent deliveries are updated accordingly. The number of removals is determined by $1 + \lfloor \frac{\theta \times it}{Max_{it}} \rfloor$, where it is the number of consecutive non-improving iterations, Max_{it} is the maximum number of consecutive non-improving iterations allowed, and θ is a parameter handling the disturbance strength. Hence, the perturbation strength increases progressively as it increases. This adaptive strategy allows the algorithm to balance intensification and diversification by exploring increasingly distant regions of the solution space when improvements become difficult to obtain. The resulting perturbed solution serves as the starting point for the next local search iteration. Since the removals typically induce stockouts, the perturbed solution is generally infeasible for the original problem. However, allowing such infeasibility enables the search to traverse infeasible regions of the solution space and escape deep local optima. Feasibility is subsequently restored during the local search.

5 Computational experiments

This section describes the computational experiments conducted to evaluate the performance of the proposed MSILS. The algorithm is implemented in C++, and all experiments were conducted on a computer with an Intel Core i7-1185G7 3.00 GHz CPU and 16GB RAM. The rest of the section is structured as follows. The test instances used for the computation experiments are summarised in Section 5.1. Section 5.3 discusses the parameter settings and demonstrates the performance gain from different components used in the MSILS. Section 5.4 evaluates the MSILS by comparing its performance with the performance of the ILS and SA algorithms proposed by Alvarez et al. (2018). This section also evaluates the performance gain from integrating the Hybrid Genetic Search (HGS) framework of Vidal (2022) into MSILS.

5.1 Test instances

To evaluate the proposed MSILS, computational experiments were conducted on benchmark instances originating from Archetti et al. (2007), Archetti et al. (2012), and the 12th DIMACS Implementation Challenge for the multi-vehicle IRP. These instances have been widely used in the IRP literature, including Archetti et al. (2012); Coelho and Laporte (2013); Desaulniers et al. (2016); Santos et al. (2016); Archetti et al. (2017b); Alvarez et al. (2018); Archetti et al. (2019); Castro et al. (2026).

The benchmark introduced by Archetti et al. (2007) contains a set of single-vehicle IRP instances comprising 50 instances ranging from 5 to 50 customers for a planning horizon of 3 time periods, and 30 instances ranging from 5 to 30 customers for a planning horizon of 6 time periods. Each instance is available under two inventory-cost scenarios (high and low), resulting in a total of 160 instances. These single-vehicle IRP instances were extended to multi-vehicle versions with a number of vehicles ranging from 2 to 5, creating an additional 640 instances. Following Alvarez et al. (2018) and the DIMACS Implementation Challenge, the vehicle capacities for the multi-vehicle IRP instances were obtained by dividing the vehicle capacity in the corresponding single-vehicle instances by the number of vehicles and rounding down to the nearest lower integer.

The 12th DIMACS Implementation Challenge further introduced 160 additional multi-vehicle instances with a number of customers ranging from 35 to 50, completing the 6-time-period planning horizon dataset. In addition, the single- and multi-vehicle IRP instances for 3 planning horizons were extended to a planning horizon of 4 and 5 periods in Archetti et al. (2017b). This was done by increasing the planning horizons from 3 to 4 and 5, resulting in another 1000 instances. Collectively, these datasets comprise 1960 instances covering 5 to 50 customers, 3 to 6 planning horizons, 1 to 5 vehicles, and two inventory-cost settings.

A larger scale of the benchmark was also considered. In Archetti et al. (2012), a set of 60 single-vehicle IRP instances was introduced with a number of customers ranging from 50 to 200. These instances were also extended to multi-vehicle instances, resulting in a total of 300 instances. The single-vehicle IRP instances from Archetti et al. (2007) and Archetti et al. (2012) can be downloaded from Single-vehicle IRP instances and the instances for the 12th DIMACS Implementation Challenge for the multi-vehicle IRP can be downloaded from DIMACS Inventory Routing Challenge.

For the above-mentioned instances, the distance c_{ij} between each node $i \in N$ and $j \in N$ is calculated and rounded to the nearest integer by

$$\left\lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + \frac{1}{2} \right\rfloor$$

, where x_i, x_j, y_i, y_j are the x- and y-coordinates for nodes i and j. Since the IRP-LR does not include inventory cost, only one inventory-cost setting was retained, and inventory-cost information was discarded. Consequently, the final

test bed consists of 980 instances for 5 to 50 customers and 150 instances for 50 to 200 customers.

Also noting, two instances were excluded from the study due to infeasibility. The instance "abs5n5_5" with 6 periods and 5 vehicles is a known infeasible instance Alvarez et al. (2018). In addition, the preliminary experiments revealed that the instance "abs4n5_5" with 5 planning periods and 5 vehicles is also infeasible, as stockouts occur even when a certain customer is visited in every period.

For reproducibility, all benchmark instances used in the computational experiments are publicly available at https://github.com/AndyElhamSarang/IRP_instances.git.

5.2 Parameter calibrations

The parameters for the MSILS are set to the values listed in Table 3, where the selected value of each parameter is underlined. The numerical parameters were calibrated with the iterated racing procedure implemented in the irace package (López-Ibáñez et al., 2016). irace repeatedly samples candidate configurations, evaluates them on a sequence of training instances, and uses the Friedman test to discard configurations that perform statistically worse than the best ones. The sampling distribution is then biased towards the surviving (elite) configurations for the next iteration.

The calibration used the large instances of Archetti et al. (2012), which have 50, 100 and 200 customers and 1 to 5 vehicles. For each of the 15 combinations of the number of customers and the number of vehicles, 7 instances were assigned to a training set and 3 different instances to a test set, giving 105 training and 45 test instances. The training instances were presented to irace in blocks of three, one instance of each size, so that every configuration was always compared on small, medium and large instances alike. Each evaluation was a single run of the MSILS with a random seed and a time limit of $Max_{time} = 60$ seconds, and the cost reported to irace was the logistic ratio of the best feasible solution found. The parameters $Max_{restart}$ and Max_{time} were fixed rather than tuned.

To keep the size of the configuration space manageable, the parameters were calibrated in three groups, keeping the other parameters fixed in each case: (i) the grid parameters $d_{\min}$ and Δ_d of the initial solution construction, with a budget of 1500 runs; (ii) the Lagrangian scalar η, with a budget of 1500 runs; and (iii) the parameters ζ, σ, ϕ and θ of the iterated local search and the perturbation, with a budget of 3000 runs. In group (iii), ζ, σ and ϕ define the maximum number of consecutive non-improving iterations, $Max_{it} = \lceil \zeta(|N'| + \sigma|T| + \phi|K|) \rceil$, and θ controls the disturbance strength (Section 4.5). The ranges searched by irace are given in the last column of Table 3; η and ζ were sampled on a logarithmic scale.

For the grid parameters, the three elite configurations were $(d_{\min}, \Delta_d) = (5, 1)$, $(3, 2)$ and $(7, 3)$. Their average logistic ratios on the 45 test instances differ by less than 0.2%, and $(3, 2)$ and $(5, 1)$ are statistically indistinguishable on the training instances. We selected $(3, 2)$, under which the grid resolution is drawn

from $\{3, 4\}$; this configuration was also ranked first in preliminary tuning runs. Compared with the previous setting $(2, 2)$, it obtained a lower logistic ratio on 12 of the 15 training instances on which both were evaluated. For η, irace converged to $\eta \approx 3.33$. However, on the training instances evaluated with both values, the difference between $\eta = 3.33$ and the initial value $\eta = 5$ was not significant (paired Wilcoxon signed-rank test, $p = 0.61$), and $\eta = 5$ was retained. For group (iii), the elite configurations agreed closely on $\zeta \in [0.51, 0.65]$, $\phi \in [14, 18]$ and $\theta \in \{1, 2\}$, whereas σ varied from 5 to 17. The best elite configuration, $\zeta = 0.62$, $\sigma = 5$, $\phi = 16$ and $\theta = 2$, was selected. Since $\zeta < 1$, each iterated local search is terminated earlier than with $\zeta = 1$, and the search restarts from a new initial solution more often. The test instances do not show which elite configuration of group (iii) is best: their average logistic ratios differ by less than 0.1%.

Overall, the calibration indicates that the MSILS is robust with respect to its numerical parameters. Poor configurations do exist: in group (iii), for example, the first statistical test discarded 34 of the 83 initial candidate configurations. Within the region of good configurations, however, the performance landscape is flat. In all three groups, Kendall's concordance coefficient between the rankings of the surviving configurations on different instances dropped to about 0.1 by the end of the race, meaning that the relative performance of these configurations is determined mainly by the instance rather than by the parameter values. Consequently, the results reported in the following sections are not sensitive to the precise parameter values in Table 3.

Table 3: Default values of parameters

Parameters	**Definition**	**Tested Range**
$d_{\min}$	Minimum grid resolution in initial solution construction	$\{1, 2, \underline{\mathbf{3}}, \ldots, 15\}$
Δ_d	Grid resolution increment, i.e., number of candidate grid resolutions in initial solution construction	$\{1, \underline{\mathbf{2}}, 3, \ldots, 8\}$
$Max_{restart}$	Number of initial starting solutions	$\{1, 2, 3, \ldots \underline{\mathbf{30}}\}$
η	Scalar for updating the Lagrange multiplier α	$\{0.1, 0.2, \ldots, \underline{\mathbf{5}}, \ldots, 10000\}$
Max_{time}	Time limit (in seconds) for MSILS run	60
ζ	Weight parameters on the maximum permissible number of consecutive iterations without improvement from the ILS	$\{0.01, 0.02, \ldots, \underline{\mathbf{0.62}}, \ldots, 5\}$
σ	Weight parameters on number of time periods to compute the maximum permissible number of consecutive iterations without improvement from the ILS	$\{0, 1, 0.2, \ldots, \underline{\mathbf{5}}, \ldots, 20\}$
ϕ	Weight parameters on number of vehicles to compute the maximum permissible number of consecutive iterations without improvement from the ILS	$\{0, 1, 0.2, \ldots, \underline{\mathbf{16}}, \ldots, 20\}$
θ	Disturbance strength	$\{0, 1, \underline{\mathbf{2}}, \ldots, 10\}$

Table 4: Variants derived from MSILS

Variants	Description
MSILS	Full algorithm described in Section 4.
$\not{R}$	Deactivating the Rebalance Procedure at line 16 in MSILS.
$D\&R$	Instead of using the Rebalance Procedure described in Section 4.4, line 16 uses the Destory_And_Rebalance procedure described in Appendix D
$\not{D}$	Deactivating the First-stage DISTURB from the procedure Perturbation.
MSILS-HGS	MSILS with HGS, with the time required for the HGS excluded from the time limit.
MSILS-Initial-Alt	Replace the proposed constructive heuristic with the constructive heuristic of Alvarez et al. (2018) MSILS-Without-Vehicle-Slack

5.3 Impact on the performance of individual mechanisms in MSILS and parameter tuning

The proposed MSILS has several key components, including an application of the Lagrangian relaxation technique, the rebalance procedure, and the disturbance procedure. This section examines the performance gain from utilising each of these components. Four variants have been derived from the MSILS. Each variant deactivates or modifies a specific component. Table 4 summarises each variant.

MSILS-Initial-Alt replaces the proposed constructive heuristic with the constructive heuristic of Alvarez et al. (2018). The procedure builds the solution period by period. In each period, customers whose inventory would become negative without a delivery must be visited and are filled up to their storage capacity, within the limit of the vehicle capacity and the supplier's available inventory; in the last period, they receive only the quantity needed to avoid the stockout. Customers expected to stock out in the next few periods may also be visited, receiving part of their remaining storage capacity. The vehicle routes for the period are then constructed with a nearest-neighbour insertion heuristic Bräysy and Gendreau (2005).

The variant $\not{L}$ is used to examine the performance gain of using the Lagrangian relaxation technique. By initialising α to a large value, the Local_Search procedure would just repair the infeasible starting solution. Thereafter, due to the high penalty for stockout, the search process would remain in the feasible solution space. All four variants and the MSILS were tested on Archettie2012 benchmark. Each version of the algorithm was given a time limit of 1 minute.

5.4 Performance evaluation

Compared with the supplementary results from Alvarez et al. (2018) available at https://www.dep.ufscar.br/munari/irp/.

The proposed MSILS is compared with ILS and SA in Alvarez et al. (2018). The algorithm is validated in small instances by comparing the optimal solutions

obtained by the ACS algorithm developed in Archetti et al. (2019).

Table 5: Average percentage gap (%) of MSILS compared with ILS

	$\|T\| = 2$	$\|T\| = 3$	$\|T\| = 4$	$\|T\| = 5$	Average
$\|N'\| = 50$	-8.00	-4.30	-5.03	-5.13	-5.61
$\|N'\| = 100$	-8.63	-6.59	-6.57	-6.90	-7.17
$\|N'\| = 200$	-9.02	-6.16	-5.16	-4.10	-6.11
Average	-8.55	-5.68	-5.58	-5.38	-6.30

Table 6: Average percentage gap (%) of MSILS compared with SA

	$\|T\| = 2$	$\|T\| = 3$	$\|T\| = 4$	$\|T\| = 5$	Average
$\|N'\| = 50$	-7.59	-5.28	-4.82	-4.97	-5.67
$\|N'\| = 100$	-10.00	-6.66	-7.56	-6.16	-7.59
$\|N'\| = 200$	-15.59	-10.18	-9.77	-7.40	-10.73
Average	-11.06	-7.37	-7.38	-6.18	-8.00

5.5 Robustness

A Functions used to construct initial solution

Find Feasible Visit Time($t_{current}$, $t_{Stockout}$, i)

while true **do**
  if $t_{current} \leq t_{Stockout}$ **then**
    // **schedule forward**//
    $T_{candidate} \leftarrow \{t \in T : t_{current} \leq t \leq t_{stockout},$
      customer i is not visited at t,
      $Q \times |K| - \sum_{k \in K} \sum_{i \in N'} q_{i,t}^k(s) > 0,$
      $S_t(s) > 0\}$
    if $T_{candidate} \neq [\,]$ **then**
      return $t \sim$ RANDOM($T_{candidate}$)
    else
      $t_{current} \leftarrow t_{Stockout} + 1$
    end if
  else
    // **schedule backward**//
    for $t \leftarrow t_{stockout}; t \geq 1; t \leftarrow t - 1$ **do**
      if customer i is not visited at time period t **then**
        if $Q \times |K| - \sum_{k \in K} \sum_{i \in N'} q_{i,t}^k(s) > 0$ **and** $S_t(s) > 0$ **then**
          return t
        end if
      end if
    end for
    return $\emptyset$
  end if
end while

Find Vehicle To Deliver Max(t, i, s)

$k' \leftarrow |K| + 1$, $q' \leftarrow 0$,
for $k \in K$ **do**
  if $t = 1$ **then**
    $q \leftarrow \max\{0, \min\{S_1(s), Q - \sum_{j \in N'} q_{j,1}^k(s), U_i - I_{i,0}\}\}$
  else
    $q \leftarrow \max\{0, \min\{S_t(s), Q - \sum_{j \in N'} q_{j,t}^k(s), U_i - I_{i,t-1}(s)\}\}$
  end if
  if $q' < q$ **then**
    $q' \leftarrow q$, $k' \leftarrow k$
  end if
end for
return k', q'

Adjust Delivery Quantities$(i,\ t,\ s)$

update $I_{0,t'}(s)$ for all $t' \in T$ with $t' \geq t$ according to (20) and (21), and $S_{t'}(s)$ for all $t' \in T$ according to (24)
update $I_{i,t}(s) - \mu_{i,t}(s)$ according to (18) if $t = 1$, and (19) otherwise
for $\tau \leftarrow t+1$ **to** $|T|$ **do**
  if customer i is visited by a vehicle k at time period τ **then**
    $q^k_{i,\tau}(s) \leftarrow \min\{q^k_{i,\tau}(s) + \min\{S_\tau(s), Q - \sum_{j \in N'} q^k_{j,\tau}(s)\},$
    $U_i - (I_{i,\tau-1}(s) - \mu_{i,\tau-1}(s))\}$
    update $I_{0,t'}(s)$ for all $t' \in T$ with $t' \geq \tau$ according to (20) and (21), and $S_{t'}(s)$ for all $t' \in T$ according to (24)
  end if
  update $I_{i,\tau}(s) - \mu_{i,\tau}(s)$ according to (19)
end for

B MSILS with HGS

Algorithm 8 MSILS with HGS in Vidal (2022)

$s^* \leftarrow$ Empty **and** $f(s^*) \leftarrow +\infty$
for $i \leftarrow 1$ **to** $Max_{restart}$ **do**
  $s \leftarrow$ Construct initial solution
  $s \leftarrow HGS(s)$ {to improve routing}
  if $f(s) < f(s^*)$ **then**
    $s^* \leftarrow s$
  end if
  Run_HGS_Or_Not $\leftarrow$ FALSE
  $it \leftarrow 0$, $\eta \leftarrow \eta'$, $u \leftarrow 0$
  while $it < Max_{it}$ **do**
    $\alpha \leftarrow \epsilon(s^*, s, 0, \eta)$
    $\bar{s} \leftarrow s$
    $s \leftarrow$ Local_Search(s, α)
    while $f_\mu(s, \alpha) \neq f_\mu(\bar{s}, \alpha)$ **do**
      $\alpha \leftarrow \epsilon(s^*, s, \alpha, \eta)$
      $\bar{s} \leftarrow s$
      $s \leftarrow$Local_Search(s, α)
    end while
    if $min\{it, u\} \geq U$ **then**
      $u \leftarrow 0$, $\eta \leftarrow \kappa \times \eta$
    end if
    if s is feasible **and** $f(s) < f(s^*)$ **then**
      $s^* \leftarrow s$, $it \leftarrow 0$
      $s^* \leftarrow$ Rebalance(s^*)
      Run_HGS_Or_Not $\leftarrow$ TRUE
    end if
    $s \leftarrow$ Disturbance(s^*, it), $it \leftarrow it + 1$, $u \leftarrow u + 1$
  end while
  if Run_HGS_Or_Not = TRUE **then**
    $s^* \leftarrow HGS(s^*)$
  end if
end for
return solution s^*

C Functions used in Rebalance procedure guarantee feasible solution

Algorithm 9 Feasible_Quantity_To_Removal(i, j)

Quantity_To_Remove $\leftarrow min\{I_{i,j}, q_{i,j}\}$
$I \leftarrow I_{i,j-1} + q_{i,j} - r_{i,j}$ −Quantity_To_Remove {New inventory level of period j}
Stockout $\leftarrow 0$
for $time \leftarrow j$ **to** $|T|$ **do**
  $I \leftarrow I_{i,time} + q_{i,j} - r_{i,j}$
  if Stockout $\leq -I'$ **then**
    Stockout $\leftarrow -I'$
  end if
end for
return $\leftarrow max\{0, Quantity - Stockout\}$

Algorithm 10 Assign_max_possible_quantity_after_feasible(s)

for $i \leftarrow 1$ **to** $|T|$ **do**
  PRIORITY $\leftarrow \emptyset$
  for $j \leftarrow 1$ **to** $|N'|$ **do**
    if customer j is visited on day i **then**
      Add customer j to PRIORITY with computed priority value
    end if
  end for
  while PRIORITY is not empty **do**
    Select a customer randomly according to PRIORITY weights
    Deliver the maximum feasible quantity to the selected customer
    Remove the selected customer from PRIORITY
    Update inventory levels and delivery quantity for subsequent days
  end while
end for
return s

D Rebalance procedure without guarantee feasible solution

Algorithm 11 Destory_And_Rebalance(s^*)

$s \leftarrow s^*$
Set all delivery quantities in s to zero
Update inventory levels in s
for $i \leftarrow 1$ **to** $|T|$ **do**
  for $j \leftarrow 1$ **to** $|N'|$ **do**
    if customer j is visited on day i **then**
      Quantity $\leftarrow$ Find_max_quantity_avoid_stockout(j, i)
      Assign Quantity
      Update inventory levels for all subsequent days
    end if
  end for
end for
Stockout_resolved $\leftarrow$ FALSE
$\{s, Stockout_resolved\} \leftarrow$Repair$(s, Stockout_resolved)$
if Stock_resolved = FALSE **then**
  return s
end if
$s \leftarrow$Assign_max_possible_quantity_after_feasible(s)
Remove visits with zero delivery quantities
return s

Algorithm 12 Find_max_quantity_avoid_stockout($customer, time$)

Quantity $\leftarrow 0$
Quantity $\leftarrow$ Find_max_stockout$(customer, time)$
Quantity $\leftarrow \min\{Quantity, U_{customer} - \max\{I_{customer,time-1}, 0\}\}$ {If stockout occurs in the previous period, assumes the inventory level is 0.}
Quantity $\leftarrow \min\{Quantity, Q - Vehicle_load\}$
return Quantity

Algorithm 13 Find_max_stockout$(customer, time)$

```
1: Largest_stockout ← 0
2: for t ← time to |T| do
3:    if Largest_stokout > I_{customer,t} then
4:       Largest_stockout ← I_{customer,t}
5:    end if
6: end for
7: Largest_stockout ← −Largest_stockout
8: return Largest_stockout
```

Algorithm 14 Repair$(s, Stockout_resolved)$

```
1: Possible_repair ← TRUE
2: while Stockout_resolved = FALSE and Possible_repair = TRUE do
3:    Stockout_resolved ← TRUE
4:    Possible_repair ← FALSE
5:    for i ← 1 to |T| do
6:       for j ← 1 to |N'| do
7:          if customer j has stockout at time i then
8:             Stockout_resolved ← FALSE
9:             Quantity_stockout ← I_{j,i}
10:            for k ← 1 to time i do
11:               if Customer j is visited in time k then
12:                  for customer z ∈ N' are visited by the same vehicle do
13:                     if Some quantity, Δ associated with customer z can be removed then
14:                        for x ← 1 to |T| do
15:                           if customer z is visited on time x then
16:                              if Δ can be delivered on time x to customer z without stockout then
17:                                 Possible_repair ← TRUE
18:                                 Quantity_Δ ← min{Quantity_stockout, Δ}
19:                                 Remove Quantity_Δ from visiting customer z on time k
20:                                 Deliver extra Quantity_Δ unit on day x
21:                                 Deliver Quantity_Δ unit for customer j on time k
22:                                 Update delivery quantity and inventory level
23:                              end if
24:                           end if
25:                        end for
26:                     end if
27:                  end for
28:               end if
29:            end for
30:         end if
31:      end for
32:   end for
33: end while
34: return {s, Stockout_resolved}
```

References

Absi, N., Cattaruzza, D., Feillet, D., Ogier, M., and Semet, F. (2020). A heuristic branch-cut-and-price algorithm for the ROADEF/EURO challenge on inventory routing. *Transportation Science*, 54:313–329.

Alonso-Pecina, F., Hernández-Báez, I. Y., López-Díaz, R. E., and Cruz-Rosales, M. (2024). Iterated local search approach to a single-product, multiple-source, inventory-routing problem. *Mathematics*, 12(7):991.

Alvarez, A., Munari, P., and Morabito, R. (2018). Iterated local search and simulated annealing algorithms for the inventory routing problem. *International Transactions in Operational Research*, 25(6):1785–1809.

André, J., Bourreau, E., and Wölfler Calvo, R. (2020). Introduction to the special section: ROADEF/EURO challenge 2016—inventory routing problem. *Transportation Science*, 54:299–301.

Archetti, C., Bertazzi, L., Hertz, A., and Speranza, M. G. (2012). A hybrid heuristic for an inventory routing problem. *INFORMS Journal on Computing*, 24(1):101–116.

Archetti, C., Bertazzi, L., Laporte, G., and Speranza, M. G. (2007). A branch-and-cut algorithm for a vendor-managed inventory-routing problem. *Transportation Science*, 41(3):382–391.

Archetti, C., Boland, N., and Grazia Speranza, M. (2017a). A matheuristic for the multivehicle inventory routing problem. *INFORMS Journal on Computing*, 29(3):377–387.

Archetti, C., Coelho, L. C., and Speranza, M. G. (2019). An exact algorithm for the inventory routing problem with logistic ratio. *Transportation Research Part E: Logistics and Transportation Review*.

Archetti, C., Desaulniers, G., and Speranza, M. G. (2017b). Minimizing the logistic ratio in the inventory routing problem. *EURO Journal on Transportation and Logistics*, 6(4):289–306.

Archetti, C. and Speranza, M. G. (2016). The inventory routing problem: the value of integration. *International Transactions in Operational Research*, 23(3):393–407.

Baldacci, R., Lim, A., Traversi, E., and Wölfler Calvo, R. (2020). Optimal solution of vehicle routing problems with fractional objective function. *Transportation Science*, 54:434–452.

Bell, W. J., Dalberto, L. M., Fisher, M. L., Greenfield, A. J., Jaikumar, R., Kedia, P., Mack, R. G., and Prutzman, P. J. (1983). Improving the distribution of industrial gases with an on-line computerized routing and scheduling optimizer. *Interfaces*, 13(6):4–23.

Benoist, T., Gardi, F., Jeanjean, A., and Estellon, B. (2011). Randomized local search for real-life inventory routing. *Transportation Science*, 45(3):381–398.

Bertazzi, L. and Speranza, M. G. (2012). Inventory routing problems: An introduction. *EURO Journal on Transportation and Logistics*, 1:307–326.

Bräysy, O. and Gendreau, M. (2005). Vehicle routing problem with time windows, part i: Route construction and local search algorithms. *Transportation Science*, 39(1):104–118.

Castro, B., Poggi, M., and Martinelli, R. (2026). A hybrid genetic search for the inventory routing problem. *Computers & Operations Research*, page 107376.

Coelho, L. C., Cordeau, J.-F., and Laporte, G. (2014). Thirty years of inventory routing. *Transportation science*, 48(1):1–19.

Coelho, L. C., De Maio, A., and Laganà, D. (2020). A variable mip neighborhood descent for the multi-attribute inventory routing problem. *Transportation Research Part E: Logistics and Transportation Review*, 144:102137.

Coelho, L. C. and Laporte, G. (2013). A branch-and-cut algorithm for the multi-product multi-vehicle inventory-routing problem. *International Journal of Production Research*, 51(23-24):7156–7169.

Desaulniers, G., Rakke, J. G., and Coelho, L. C. (2016). A branch-price-and-cut algorithm for the inventory-routing problem. *Transportation Science*, 50(3):1060–1076.

Dinkelbach, W. (1967). On nonlinear fractional programming. *Management Science*, 13(7):492–498.

Ekşi, A., Elyasi, M., Minner, S., Van Woensel, T., Özener, O. Ö., and Ekici, A. (2026). Inventory-routing problems: Review, challenges, and innovations. *European Journal of Operational Research*.

Elshaer, R. and Awad, H. (2020). A taxonomic review of metaheuristic algorithms for solving the vehicle routing problem and its variants. *Computers & Industrial Engineering*, 140:106242.

Fisher, M. L. (1981). The lagrangian relaxation method for solving integer programming problems. *Management Science*, 27(1):1–18.

He, Y., Artigues, C., Briand, C., Jozefowiez, N., and Ngueveu, S. U. (2020). A matheuristic with fixed-sequence reoptimization for a real-life inventory routing problem. *Transportation Science*, 54(2):355–374.

Iswari, T., Caris, A., and Braekers, K. (2026). Extending the inventory routing problem to support integrated decision-making in an urban distribution network. *Networks*, 87(3):289–311.

Kheiri, A. (2020). Heuristic sequence selection for inventory routing problem. *Transportation Science*, 54:302–312.

Kindervater, G. A. P. and Savelsbergh, M. W. P. (1997). Vehicle routing: handling edge exchanges. *Local Search in Combinatorial Optimization*, pages 337–360.

Laganà, D., Malaguti, E., Monaci, M., Musmanno, R., and Paronuzzi, P. (2024). An iterated local search matheuristic approach for the multi-vehicle inventory routing problem. *Computers & Operations Research*, 169:106717.

Laporte, G., Gendreau, M., Potvin, J. Y., and Semet, F. (2000). Classical and modern heuristics for the vehicle routing problem. *International Transactions in Operational Research*, 7(4-5):285–300.

López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L. P., Birattari, M., and Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58.

Man, X., Chang, D., Peng, Z., and Han, W. (2025). A logistics ratio-optimised inventory routing problem with lateral transhipment using mixed-integer robust fractional programming. *International Journal of Production Research*, 63:10381–10412.

Moin, N. H. and Salhi, S. (2007). Inventory routing problems: A logistical overview. *Journal of the Operational Research Society*, 58:1185–1194.

Santos, E., Ochi, L. S., Simonetti, L., and González, P. H. (2016). A hybrid heuristic based on iterated local search for multivehicle inventory routing problem. *Electronic Notes in Discrete Mathematics*, 52:197–204.

Singh, T., Arbogast, J. E., and Neagu, N. (2015). An incremental approach using local-search heuristic for inventory routing problem in industrial gases. *Computers & Chemical Engineering*, 80:199–210.

Skålnes, J., Vadseth, S. T., Andersson, H., and Stålhane, M. (2023). A branch-and-cut embedded matheuristic for the inventory routing problem. *Computers & Operations Research*, 159:106353.

Sofianopoulou, S. and Mitsopoulos, I. (2021). A review and classification of heuristic algorithms for the inventory routing problem. *International Journal of Operational Research*.

Su, Z., Lü, Z., Wang, Z., Qi, Y., and Benlic, U. (2020). A matheuristic algorithm for the inventory routing problem. *Transportation Science*, 54(2):330–354.

Vadseth, S. T., Andersson, H., Stålhane, M., and Chitsaz, M. (2023). A multi-start route improving matheuristic for the production routeing problem. *International Journal of Production Research*, 61(22):7608–7629.

Vidal, T. (2022). Hybrid genetic search for the cvrp: Open-source implementation and swap* neighborhood. *Computers & Operations Research*, 140:105643.

Zhao, J., Archetti, C., Pham, T. A., and Vidal, T. (2025). Large neighborhood and hybrid genetic search for inventory routing problems. *European Journal of Operational Research*.